JavaScript

Object-Oriented Programming

Complete Reference Guide

From Fundamentals to Advanced Patterns

Chapter 1: Introduction to Object-Oriented Programming

Object-Oriented Programming (OOP) is a programming paradigm — a style of writing code — that organizes software around objects rather than functions and logic. Instead of thinking about what your program does step by step, you think about what things (objects) exist in your program and how they interact with each other.

JavaScript is a multi-paradigm language, which means it supports multiple styles of coding — procedural, functional, and object-oriented. Understanding OOP in JavaScript is essential for writing scalable, reusable, and maintainable code.

1.1 Why OOP?

Consider building a large application without OOP. You would end up with hundreds of unrelated variables and functions scattered across your codebase — hard to manage, hard to debug, and nearly impossible to extend. OOP solves this by:

- Grouping related data and behaviour together into a single unit (an object)
- Hiding internal details and exposing only what is needed (encapsulation)
- Sharing common behaviour without duplicating code (inheritance)
- Allowing one interface to work with many different types (polymorphism)

1.2 The Four Pillars of OOP

Encapsulation

Bundling data (properties) and the methods that operate on that data inside a single unit, and restricting direct access from the outside.

Abstraction

Hiding complex implementation details and exposing only a simple, clean interface to the user of the object.

Inheritance

Allowing a new class to acquire the properties and methods of an existing class, promoting code reuse.

Polymorphism

Allowing objects of different types to be treated through the same interface, each responding in its own way.

Chapter 2: Objects in JavaScript

An object is a collection of key-value pairs (properties) where values can be data or functions (methods). Everything in JavaScript, except primitive values, is either an object or can behave like one.

2.1 Object Literal Syntax

The simplest way to create an object in JavaScript is using the object literal notation — curly braces containing key-value pairs.

```
// Creating an object using literal syntax
const person = {
  name: "Alice",
  age: 28,
  city: "Mumbai",

  // Method — a function stored as a property
  greet() {
    return `Hello, I am ${this.name} from ${this.city}.`;
  },

  introduce() {
    console.log(`I am ${this.age} years old.`);
  }
};

console.log(person.name);        // "Alice"
console.log(person["age"]);      // 28  (bracket notation)
console.log(person.greet());     // "Hello, I am Alice from Mumbai."
person.introduce();              // "I am 28 years old."
```

2.2 Accessing and Modifying Properties

You can read, update, add, or delete properties on any object at any time.

```
const car = { brand: "Toyota", model: "Camry", year: 2020 };

// Read
console.log(car.brand);          // "Toyota"

// Update
car.year = 2023;
console.log(car.year);           // 2023

// Add a new property dynamically
car.color = "Silver";
console.log(car.color);          // "Silver"

// Delete a property
delete car.model;
console.log(car.model);          // undefined

// Check if property exists
console.log("brand" in car);     // true
console.log("model" in car);     // false
```

2.3 The "this" Keyword

Inside a method, "this" refers to the object that the method belongs to. It allows methods to access and operate on the object's own properties.

```
const bank = {
  owner: "Rahul",
  balance: 5000,

  deposit(amount) {
    this.balance += amount;   // "this" refers to the bank object
    console.log(`Deposited ₹${amount}. New balance: ₹${this.balance}`);
  },

  withdraw(amount) {
    if (amount > this.balance) {
      console.log("Insufficient funds!");
    } else {
      this.balance -= amount;
      console.log(`Withdrew ₹${amount}. Remaining: ₹${this.balance}`);
    }
  }
};

bank.deposit(1000);    // Deposited ₹1000. New balance: ₹6000
bank.withdraw(2000);   // Withdrew ₹2000. Remaining: ₹4000
bank.withdraw(9999);   // Insufficient funds!
```

> **Note:** *Arrow functions do NOT have their own "this". They inherit "this" from the surrounding lexical scope. Never use arrow functions as methods in objects if you need to access "this".*

2.4 Object Destructuring

Destructuring lets you extract values from objects into individual variables cleanly and concisely.

```
const student = { name: "Priya", grade: "A", score: 95, city: "Delhi" };

// Without destructuring (old way)
const name1 = student.name;
const grade1 = student.grade;

// With destructuring (modern way)
const { name, grade, score } = student;
console.log(name, grade, score);   // "Priya" "A" 95

// Rename while destructuring
const { name: studentName, city: studentCity } = student;
console.log(studentName);          // "Priya"

// Default values
const { subject = "Math" } = student;
console.log(subject);              // "Math" (property did not exist)

// In function parameters
function displayStudent({ name, score }) {
```

```
  console.log(`${name} scored ${score}`);
}
displayStudent(student);           // "Priya scored 95"
```

Chapter 3: Constructor Functions

When you need to create many objects with the same structure, writing object literals for each one is repetitive. Constructor functions are a blueprint — a regular function called with the "new" keyword to produce new objects.

3.1 How Constructor Functions Work

```
// Constructor function — by convention, name starts with capital letter
function Person(name, age, city) {
  this.name = name;   // "this" is the newly created object
  this.age = age;
  this.city = city;

  this.greet = function() {
    return `Hi, I am ${this.name} from ${this.city}.`;
  };
}

// "new" creates a blank object, sets "this" to it, runs the function,
// then returns the object automatically
const p1 = new Person("Arun", 25, "Chennai");
const p2 = new Person("Meera", 30, "Bangalore");

console.log(p1.name);      // "Arun"
console.log(p2.greet());   // "Hi, I am Meera from Bangalore."
console.log(p1 instanceof Person);  // true
```

> **Note:** *The "new" keyword does four things behind the scenes: (1) Creates a new empty object, (2) Sets "this" to that object, (3) Runs the constructor function body, (4) Returns the object automatically.*

3.2 The Problem with Constructor Functions

Every time you create a new object with a constructor function, the methods defined inside it (like greet) are duplicated in memory for each instance. If you create 1,000 Person objects, you get 1,000 copies of the same greet function — wasteful! The solution is the prototype, covered in the next chapter.

Chapter 4: Prototypes & the Prototype Chain

Prototypes are the mechanism by which JavaScript objects inherit features from one another. Every JavaScript object has a hidden internal link to another object called its prototype. When you access a property or method on an object, JavaScript first looks at the object itself. If it does not find it there, it looks at the prototype. If not there, it looks at the prototype's prototype — and so on. This chain is called the Prototype Chain.

4.1 Adding Methods to the Prototype

Instead of defining methods inside the constructor (which copies them per instance), define them on the constructor's prototype. All instances then share a single copy of the method.

```
function Animal(name, sound) {
  this.name = name;
  this.sound = sound;
}

// Add method to prototype — shared across ALL Animal instances
Animal.prototype.speak = function() {
  return `${this.name} says: ${this.sound}!`;
};

Animal.prototype.describe = function() {
  return `I am an animal named ${this.name}.`;
};

const dog = new Animal("Dog", "Woof");
const cat = new Animal("Cat", "Meow");

console.log(dog.speak());       // "Dog says: Woof!"
console.log(cat.speak());       // "Cat says: Meow!"

// Both share ONE speak function in memory (not two copies)
console.log(dog.speak === cat.speak);   // true
```

4.2 Understanding the Prototype Chain

```
const obj = { x: 10 };

// obj -> Object.prototype -> null

console.log(obj.x);           // 10 (found on obj itself)
console.log(obj.toString());  // "[object Object]" (found on Object.prototype)
console.log(obj.unknown);     // undefined (not found anywhere in chain)

// Visualising the chain:
// obj  --> { x: 10 }
//            [[Prototype]]
//               |
//         Object.prototype  --> { toString, hasOwnProperty, ... }
//               |
//             null  (end of chain)

// Check if property is own (not inherited)
```

```
console.log(obj.hasOwnProperty("x"));          // true
console.log(obj.hasOwnProperty("toString"));   // false (inherited)
```

4.3 Object.create() — Manual Prototype Linking

Object.create() lets you create an object with a specific prototype, giving you explicit control over inheritance.

```
const vehicleProto = {
  start() { return `${this.brand} engine started.`; },
  stop()  { return `${this.brand} engine stopped.`; }
};

// Create an object whose prototype IS vehicleProto
const myCar = Object.create(vehicleProto);
myCar.brand = "Honda";
myCar.model = "City";

console.log(myCar.start());    // "Honda engine started."
console.log(myCar.stop());     // "Honda engine stopped."

// myCar does not own start/stop — they come from vehicleProto
console.log(myCar.hasOwnProperty("start"));  // false
console.log(myCar.hasOwnProperty("brand"));  // true

// Check prototype
console.log(Object.getPrototypeOf(myCar) === vehicleProto);  // true
```

Chapter 5: Classes (ES6+)

ES6 (2015) introduced the class keyword, providing a cleaner, more intuitive syntax for OOP in JavaScript. Classes are essentially syntactic sugar over the prototype-based system — under the hood, they work the same way, but the code looks much more like traditional OOP languages such as Java or C++.

5.1 Basic Class Syntax

```
class Person {
  // constructor runs automatically when "new Person()" is called
  constructor(name, age) {
    this.name = name;
    this.age  = age;
  }

  // Method — automatically placed on Person.prototype
  greet() {
    return `Hello! I am ${this.name}, ${this.age} years old.`;
  }

  haveBirthday() {
    this.age++;
    console.log(`Happy Birthday ${this.name}! You are now ${this.age}.`);
  }
}

const p = new Person("Kiran", 22);
console.log(p.greet());      // "Hello! I am Kiran, 22 years old."
p.haveBirthday();            // "Happy Birthday Kiran! You are now 23."
console.log(p.age);         // 23
```

5.2 Getters and Setters

Getters and setters let you define computed properties and add validation logic when reading or writing a property. They look like properties when accessed but are actually methods.

```
class Temperature {
  constructor(celsius) {
    this._celsius = celsius;  // "_" prefix is a convention for "private-ish"
  }

  // Getter — accessed like a property: temp.fahrenheit
  get fahrenheit() {
    return (this._celsius * 9) / 5 + 32;
  }

  // Setter — called when you assign: temp.fahrenheit = 98.6
  set fahrenheit(value) {
    this._celsius = ((value - 32) * 5) / 9;
  }

  get celsius() { return this._celsius; }

  set celsius(value) {
    if (value < -273.15) throw new Error("Temperature below absolute zero!");
```

```
    this._celsius = value;
  }
}

const temp = new Temperature(100);
console.log(temp.fahrenheit);      // 212 (100°C = 212°F)
temp.fahrenheit = 32;              // set via setter
console.log(temp.celsius);         // 0

temp.celsius = 25;
console.log(temp.fahrenheit);      // 77
```

5.3 Static Methods and Properties

Static methods belong to the class itself, not to instances. They are useful for utility functions that are logically related to the class but do not need access to any instance data.

```
class MathUtils {
  static PI = 3.14159265;   // Static property

  static add(a, b) { return a + b; }
  static multiply(a, b) { return a * b; }

  static circleArea(radius) {
    return MathUtils.PI * radius * radius;
  }

  static isPrime(n) {
    if (n < 2) return false;
    for (let i = 2; i <= Math.sqrt(n); i++) {
      if (n % i === 0) return false;
    }
    return true;
  }
}

// Called on the CLASS, not on instances
console.log(MathUtils.add(5, 3));           // 8
console.log(MathUtils.circleArea(7));       // 153.938...
console.log(MathUtils.isPrime(17));         // true
console.log(MathUtils.isPrime(18));         // false

// Attempting on instance throws error
// const m = new MathUtils();
// m.add(1, 2);  // TypeError!
```

Chapter 6: Encapsulation & Private Fields

Encapsulation means hiding the internal state of an object and only allowing modification through controlled methods. This prevents accidental corruption of data from outside the class.

6.1 Private Fields with # (ES2022)

Private fields are declared with a # prefix and are truly inaccessible from outside the class. This is the modern, recommended way to implement encapsulation in JavaScript.

```
class BankAccount {
  #balance;          // private field — cannot be accessed outside
  #owner;
  #transactionLog = [];  // private with default value

  constructor(owner, initialBalance) {
    this.#owner   = owner;
    this.#balance = initialBalance;
  }

  deposit(amount) {
    if (amount <= 0) throw new Error("Amount must be positive.");
    this.#balance += amount;
    this.#transactionLog.push(`Deposited: ₹${amount}`);
    return this;   // enables method chaining
  }

  withdraw(amount) {
    if (amount > this.#balance) throw new Error("Insufficient funds.");
    this.#balance -= amount;
    this.#transactionLog.push(`Withdrew: ₹${amount}`);
    return this;
  }

  get balance() { return this.#balance; }
  get owner()   { return this.#owner; }

  printStatement() {
    console.log(`--- Account: ${this.#owner} ---`);
    this.#transactionLog.forEach(t => console.log(t));
    console.log(`Balance: ₹${this.#balance}`);
  }
}

const acc = new BankAccount("Sunita", 10000);
acc.deposit(2000).deposit(500).withdraw(1500);  // method chaining
acc.printStatement();

// console.log(acc.#balance);  // SyntaxError — private!
console.log(acc.balance);   // 11000 — accessed via getter
```

6.2 Private Methods

```
class PasswordManager {
  #password;
```

```
  constructor(password) {
    this.#password = this.#hash(password);  // use private method
  }

  #hash(password) {
    // Simple simulation (real hashing uses bcrypt etc.)
    return password.split("").reverse().join("") + "_hashed";
  }

  verify(inputPassword) {
    return this.#hash(inputPassword) === this.#password;
  }

  changePassword(old, newPass) {
    if (!this.verify(old)) throw new Error("Wrong password!");
    this.#password = this.#hash(newPass);
    console.log("Password changed successfully.");
  }
}

const pm = new PasswordManager("mySecret123");
console.log(pm.verify("mySecret123"));   // true
console.log(pm.verify("wrongPass"));     // false
pm.changePassword("mySecret123", "newPass456");
```

Chapter 7: Inheritance

Inheritance allows a child class to inherit properties and methods from a parent class. This promotes code reuse — you define common behaviour once in the parent and specialise it in child classes.

7.1 The "extends" Keyword and "super"

"extends" creates a child class that inherits from a parent. "super()" calls the parent's constructor. "super.method()" calls a parent's method from a child.

```
class Animal {
  constructor(name, age) {
    this.name = name;
    this.age  = age;
  }

  eat() {
    return `${this.name} is eating.`;
  }

  sleep() {
    return `${this.name} is sleeping.`;
  }

  toString() {
    return `Animal(${this.name}, age ${this.age})`;
  }
}

// Dog inherits from Animal
class Dog extends Animal {
  constructor(name, age, breed) {
    super(name, age);  // MUST call super() before using "this"
    this.breed = breed;
  }

  bark() {
    return `${this.name} barks: Woof!`;
  }

  // Override parent method
  toString() {
    return `Dog(${this.name}, ${this.breed}, age ${this.age})`;
  }
}

class Cat extends Animal {
  constructor(name, age, indoor) {
    super(name, age);
    this.indoor = indoor;
  }

  purr() { return `${this.name} purrs...`; }
}

const dog = new Dog("Rocky", 3, "Labrador");
const cat = new Cat("Whiskers", 5, true);
```

```
console.log(dog.eat());       // "Rocky is eating."   (inherited)
console.log(dog.bark());      // "Rocky barks: Woof!" (own method)
console.log(dog.toString());  // "Dog(Rocky, Labrador, age 3)"

console.log(dog instanceof Dog);     // true
console.log(dog instanceof Animal);  // true
console.log(cat instanceof Dog);     // false
```

7.2 Multi-level Inheritance

JavaScript supports multi-level inheritance — a child can extend a class that itself extends another class. The prototype chain handles lookup through all levels.

```
class Shape {
  constructor(color) { this.color = color; }
  getColor() { return `Color: ${this.color}`; }
  area()     { return 0; }   // to be overridden
}

class Rectangle extends Shape {
  constructor(color, width, height) {
    super(color);
    this.width  = width;
    this.height = height;
  }
  area()      { return this.width * this.height; }
  perimeter() { return 2 * (this.width + this.height); }
}

class Square extends Rectangle {
  constructor(color, side) {
    super(color, side, side);  // Rectangle gets side as both width and height
    this.side = side;
  }
  describe() {
    return `Square: side=${this.side}, area=${this.area()},
color=${this.color}`;
  }
}

const sq = new Square("blue", 5);
console.log(sq.describe());     // "Square: side=5, area=25, color=blue"
console.log(sq.perimeter());    // 20 (from Rectangle)
console.log(sq.getColor());     // "Color: blue" (from Shape)

console.log(sq instanceof Square);     // true
console.log(sq instanceof Rectangle);  // true
console.log(sq instanceof Shape);      // true
```

7.3 Calling Parent Methods with super

```
class Logger {
  log(message) {
    console.log(`[LOG] ${message}`);
  }
}
```

```
class TimestampLogger extends Logger {
  log(message) {
    const time = new Date().toISOString();
    super.log(`${time} — ${message}`);  // call parent log then extend it
  }
}

class PrefixLogger extends TimestampLogger {
  constructor(prefix) {
    super();
    this.prefix = prefix;
  }
  log(message) {
    super.log(`[${this.prefix}] ${message}`);
  }
}

const logger = new PrefixLogger("ERROR");
logger.log("Something went wrong!");
// [LOG] 2025-..T..Z — [ERROR] Something went wrong!
```

Chapter 8: Polymorphism

Polymorphism means "many forms". In OOP it means that different classes can be treated through the same interface (same method name), but each class implements that method differently. This enables you to write flexible, general-purpose code.

8.1 Method Overriding

When a child class defines a method with the same name as a parent method, it overrides it. The most specific (child) version is called.

```
class Payment {
  process(amount) {
    return `Processing ₹${amount}...`;  // generic
  }
}

class CreditCard extends Payment {
  process(amount) {
    return `Credit Card: Charging ₹${amount} to card ending 4242.`;
  }
}

class UPI extends Payment {
  constructor(upiId) {
    super();
    this.upiId = upiId;
  }
  process(amount) {
    return `UPI: Sending ₹${amount} via ${this.upiId}.`;
  }
}

class NetBanking extends Payment {
  process(amount) {
    return `Net Banking: Transfer of ₹${amount} initiated.`;
  }
}

// Polymorphism in action — same interface, different behaviour
const methods = [
  new CreditCard(),
  new UPI("priya@upi"),
  new NetBanking()
];

methods.forEach(method => {
  console.log(method.process(500));
});
// Credit Card: Charging ₹500 to card ending 4242.
// UPI: Sending ₹500 via priya@upi.
// Net Banking: Transfer of ₹500 initiated.
```

> **Note:** *This is the power of polymorphism — the forEach loop does not care what type of Payment it has. It just calls .process() and each object responds appropriately.*

8.2 Duck Typing Polymorphism

JavaScript uses "duck typing" — if an object has the required method, it works, regardless of its class. "If it walks like a duck and quacks like a duck, it is a duck."

```
// These are completely unrelated classes — no common parent!
class Robot {
  makeSound() { return "Beep boop beep!"; }
}

class Human {
  makeSound() { return "Hello there!"; }
}

class Dog {
  makeSound() { return "Woof woof!"; }
}

// Works with ANY object that has makeSound()
function makeNoise(entity) {
  console.log(entity.makeSound());
}

makeNoise(new Robot());   // "Beep boop beep!"
makeNoise(new Human());   // "Hello there!"
makeNoise(new Dog());     // "Woof woof!"
makeNoise({ makeSound: () => "Custom noise!" });  // works too!
```

Chapter 9: Abstraction

Abstraction means hiding complexity. You expose only what is necessary and keep implementation details hidden. Users of your class should not need to know HOW it works internally — only WHAT it does.

9.1 Abstract Classes (Simulated in JavaScript)

JavaScript does not have a built-in "abstract" keyword, but we can simulate abstract classes by throwing errors in methods that must be overridden.

```
class DatabaseConnection {
  constructor(host) {
    if (new.target === DatabaseConnection) {
      throw new Error("Cannot instantiate abstract class DatabaseConnection!");
    }
    this.host = host;
    this.connected = false;
  }

  // Abstract methods — subclasses MUST implement these
  connect()    { throw new Error("connect() must be implemented."); }
  disconnect() { throw new Error("disconnect() must be implemented."); }
  query(sql)   { throw new Error("query() must be implemented."); }

  // Concrete method — available to all subclasses
  isConnected() {
    return this.connected;
  }
}

class MySQLConnection extends DatabaseConnection {
  connect()    { this.connected = true; console.log(`MySQL connected to
${this.host}`); }
  disconnect() { this.connected = false; console.log("MySQL disconnected."); }
  query(sql)   { return `MySQL result for: ${sql}`; }
}

class MongoDBConnection extends DatabaseConnection {
  connect()    { this.connected = true; console.log(`MongoDB connected to
${this.host}`); }
  disconnect() { this.connected = false; console.log("MongoDB disconnected.");
}
  query(sql)   { return `MongoDB result for: ${sql}`; }
}

const db = new MySQLConnection("localhost:3306");
db.connect();
console.log(db.query("SELECT * FROM users"));
console.log(db.isConnected());    // true
db.disconnect();

// new DatabaseConnection("host");  // Error: Cannot instantiate abstract class
```

Chapter 10: Mixins & Composition

JavaScript does not support multiple inheritance (a class cannot extend two parents). Mixins are a pattern that lets you add capabilities from multiple sources to a class, achieving multiple inheritance safely.

10.1 Mixin Pattern

```
// Mixins are plain objects containing methods
const Serializable = {
  toJSON() {
    return JSON.stringify(this);
  },
  fromJSON(jsonStr) {
    return JSON.parse(jsonStr);
  }
};

const Printable = {
  print() {
    console.log(`[PRINT] ${JSON.stringify(this)}`);
  }
};

const Validatable = {
  validate() {
    return Object.keys(this).every(key => this[key] !== null && this[key] !==
undefined);
  }
};

// Apply mixins to a class
class User {
  constructor(name, email) {
    this.name  = name;
    this.email = email;
  }
}

// Copy methods from mixins into User.prototype
Object.assign(User.prototype, Serializable, Printable, Validatable);

const user = new User("Ankit", "ankit@example.com");
user.print();                    // [PRINT] {"name":"Ankit","email":"..."}
console.log(user.toJSON());      // '{"name":"Ankit","email":"..."}'
console.log(user.validate());    // true
```

10.2 Composition over Inheritance

"Favour composition over inheritance" is a widely accepted principle. Instead of building deep class hierarchies, compose objects from smaller, focused capabilities.

```
// Small, reusable behaviour factories
const canFly = (base) => ({
  ...base,
  fly() { return `${base.name} is flying!`; }
});
```

```
const canSwim = (base) => ({
  ...base,
  swim() { return `${base.name} is swimming!`; }
});

const canRun = (base) => ({
  ...base,
  run() { return `${base.name} is running!`; }
});

// Compose abilities without any class hierarchy
const duck = canFly(canSwim(canRun({ name: "Duck" })));
const eagle = canFly(canRun({ name: "Eagle" }));
const fish  = canSwim({ name: "Fish" });

console.log(duck.fly());    // "Duck is flying!"
console.log(duck.swim());   // "Duck is swimming!"
console.log(duck.run());    // "Duck is running!"
console.log(eagle.fly());   // "Eagle is flying!"
console.log(fish.swim());   // "Fish is swimming!"
// console.log(fish.fly());  // TypeError — fish cannot fly (correct!)
```

Chapter 11: Property Descriptors & Object Control

Every property on an object has hidden metadata called a property descriptor, which controls how that property behaves. Understanding descriptors lets you create truly immutable objects and finely control data.

11.1 Property Descriptors

```
const obj = {};

Object.defineProperty(obj, "pi", {
  value: 3.14159,
  writable: false,      // cannot be changed
  enumerable: true,     // appears in for...in and Object.keys()
  configurable: false   // cannot be deleted or redefined
});

console.log(obj.pi);    // 3.14159
obj.pi = 999;           // silently fails in non-strict mode
console.log(obj.pi);    // still 3.14159

// View any property's descriptor
console.log(Object.getOwnPropertyDescriptor(obj, "pi"));
// { value: 3.14159, writable: false, enumerable: true, configurable: false }

// Define multiple at once
const config = {};
Object.defineProperties(config, {
  HOST: { value: "localhost", writable: false, enumerable: true },
  PORT: { value: 3000,       writable: false, enumerable: true }
});
console.log(config.HOST, config.PORT);  // "localhost" 3000
```

11.2 Object.freeze() and Object.seal()

```
// Object.freeze() — completely immutable
const settings = Object.freeze({
  theme: "dark",
  language: "en",
  version: "2.0.1"
});

settings.theme = "light";    // silently fails
settings.newProp = "value";  // silently fails
delete settings.version;     // silently fails
console.log(settings.theme); // still "dark"
console.log(Object.isFrozen(settings));  // true

// Object.seal() — can change existing values but cannot add/delete
const user = Object.seal({
  name: "Ravi",
  age: 30
});

user.name = "Rajesh";  // OK — can update existing
user.email = "r@e.com"; // fails — cannot add new
```

```
delete user.age;         // fails — cannot delete
console.log(user);       // { name: "Rajesh", age: 30 }
console.log(Object.isSealed(user));   // true
```

Chapter 12: Symbols in OOP

Symbols are a primitive data type in JavaScript (ES6+) that produce unique, immutable values. They are primarily used as unique property keys to avoid naming collisions — especially useful in OOP when writing libraries or frameworks.

12.1 Creating and Using Symbols

```
// Every Symbol() call creates a UNIQUE value
const id1 = Symbol("id");
const id2 = Symbol("id");
console.log(id1 === id2);   // false — always unique

const USER_ID = Symbol("userId");

class User {
  constructor(name, id) {
    this.name    = name;
    this[USER_ID] = id;  // symbol as key — hidden from normal iteration
  }

  getId() { return this[USER_ID]; }
}

const user = new User("Deepa", 42);
console.log(user.name);       // "Deepa"
console.log(user.getId());    // 42

// Symbol keys are NOT visible in normal object iteration
console.log(Object.keys(user));    // ["name"] — no USER_ID!
console.log(JSON.stringify(user)); // {"name":"Deepa"} — no USER_ID!

// Access symbol keys with:
console.log(Object.getOwnPropertySymbols(user));  // [Symbol(userId)]
```

12.2 Well-Known Symbols (Custom Behaviour)

JavaScript has built-in "well-known" symbols that let you customise how your objects interact with language features like for...of, spread operators, and template literals.

```
class NumberRange {
  constructor(start, end) {
    this.start = start;
    this.end   = end;
  }

  // Symbol.iterator — makes the object usable in for...of
  [Symbol.iterator]() {
    let current = this.start;
    const end   = this.end;
    return {
      next() {
        if (current <= end) {
          return { value: current++, done: false };
        }
        return { value: undefined, done: true };
```

```
      }
    };
  }

  // Symbol.toPrimitive — controls type conversion
  [Symbol.toPrimitive](hint) {
    if (hint === "number") return this.end - this.start;
    if (hint === "string") return `Range(${this.start}-${this.end})`;
    return true;
  }
}

const range = new NumberRange(1, 5);

for (const n of range) {
  process.stdout.write(n + " ");   // 1 2 3 4 5
}

console.log([...range]);            // [1, 2, 3, 4, 5]
console.log(`${range}`);            // "Range(1-5)"
console.log(+range);                // 4 (5 - 1)
```

Chapter 13: Common OOP Design Patterns

Design patterns are proven, reusable solutions to commonly occurring problems in software design. They are not code you copy-paste — they are templates describing how to solve a problem.

13.1 Singleton Pattern

Ensures that a class has only one instance and provides a global point of access to it. Useful for things like configuration managers, database connections, or loggers.

```
class ConfigManager {
  static #instance = null;
  #config = {};

  constructor() {
    if (ConfigManager.#instance) {
      return ConfigManager.#instance;
    }
    ConfigManager.#instance = this;
  }

  set(key, value) { this.#config[key] = value; }
  get(key)        { return this.#config[key]; }

  static getInstance() {
    if (!ConfigManager.#instance) {
      new ConfigManager();
    }
    return ConfigManager.#instance;
  }
}

const c1 = ConfigManager.getInstance();
const c2 = ConfigManager.getInstance();

c1.set("theme", "dark");
console.log(c2.get("theme"));   // "dark" — same instance!
console.log(c1 === c2);          // true
```

13.2 Factory Pattern

A factory is a method or class that creates objects without specifying the exact class. The caller does not need to know which subclass will be instantiated.

```
class Circle {
  constructor(radius)  { this.radius = radius; }
  area()     { return Math.PI * this.radius ** 2; }
  describe(){ return `Circle with radius ${this.radius}`; }
}

class Rectangle {
  constructor(w, h) { this.w = w; this.h = h; }
  area()     { return this.w * this.h; }
  describe(){ return `Rectangle ${this.w}x${this.h}`; }
}
```

```
class Triangle {
  constructor(base, height) { this.base = base; this.height = height; }
  area()     { return 0.5 * this.base * this.height; }
  describe(){ return `Triangle base=${this.base}, height=${this.height}`; }
}

// Factory — decides which class to create
class ShapeFactory {
  static create(type, ...args) {
    switch (type) {
      case "circle":    return new Circle(...args);
      case "rectangle": return new Rectangle(...args);
      case "triangle":  return new Triangle(...args);
      default: throw new Error(`Unknown shape: ${type}`);
    }
  }
}

const shapes = [
  ShapeFactory.create("circle", 5),
  ShapeFactory.create("rectangle", 4, 6),
  ShapeFactory.create("triangle", 3, 8)
];

shapes.forEach(s => {
  console.log(`${s.describe()} — area: ${s.area().toFixed(2)}`);
});
```

13.3 Observer Pattern

The Observer pattern defines a one-to-many relationship. When one object (the Subject/Publisher) changes state, all its dependents (Observers/Subscribers) are notified automatically.

```
class EventEmitter {
  #events = {};

  on(event, listener) {
    if (!this.#events[event]) this.#events[event] = [];
    this.#events[event].push(listener);
    return this;  // chaining
  }

  off(event, listener) {
    if (!this.#events[event]) return;
    this.#events[event] = this.#events[event].filter(l => l !== listener);
    return this;
  }

  emit(event, ...args) {
    if (!this.#events[event]) return;
    this.#events[event].forEach(listener => listener(...args));
    return this;
  }
}

class Store extends EventEmitter {
```

```
  #state;
  constructor(initialState) {
    super();
    this.#state = initialState;
  }

  setState(newState) {
    const oldState = this.#state;
    this.#state = { ...this.#state, ...newState };
    this.emit("change", this.#state, oldState);
  }

  getState() { return this.#state; }
}

const store = new Store({ user: null, theme: "light" });

store.on("change", (newState, oldState) => {
  console.log("State changed:", newState);
});

store.setState({ user: "Vikram" });
// State changed: { user: "Vikram", theme: "light" }

store.setState({ theme: "dark" });
// State changed: { user: "Vikram", theme: "dark" }
```

13.4 Builder Pattern

The Builder pattern constructs complex objects step by step. It is perfect when an object requires many optional configuration parameters.

```
class QueryBuilder {
  #table = "";
  #conditions = [];
  #fields = ["*"];
  #orderBy = null;
  #limitVal = null;

  from(table)        { this.#table = table; return this; }
  select(...fields)  { this.#fields = fields; return this; }
  where(condition)   { this.#conditions.push(condition); return this; }
  order(field, dir = "ASC") { this.#orderBy = `${field} ${dir}`; return this; }
  limit(n)           { this.#limitVal = n; return this; }

  build() {
    let query = `SELECT ${this.#fields.join(", ")} FROM ${this.#table}`;
    if (this.#conditions.length)
      query += ` WHERE ${this.#conditions.join(" AND ")}`;
    if (this.#orderBy)
      query += ` ORDER BY ${this.#orderBy}`;
    if (this.#limitVal !== null)
      query += ` LIMIT ${this.#limitVal}`;
    return query;
  }
}

const query = new QueryBuilder()
```

```
  .from("users")
  .select("id", "name", "email")
  .where("age > 18")
  .where("active = true")
  .order("name")
  .limit(10)
  .build();

console.log(query);
// SELECT id, name, email FROM users
// WHERE age > 18 AND active = true
// ORDER BY name ASC LIMIT 10
```

Chapter 14: Iterators & Generators in OOP

Iterators and generators let your custom classes integrate with JavaScript's built-in iteration protocol — enabling for...of loops, spread syntax, and destructuring to work seamlessly with your own data structures.

14.1 Implementing a Custom Iterator

```
class LinkedList {
  #head = null;
  #size = 0;

  push(value) {
    this.#head = { value, next: this.#head };
    this.#size++;
    return this;
  }

  get size() { return this.#size; }

  // Symbol.iterator makes LinkedList iterable
  [Symbol.iterator]() {
    let current = this.#head;
    return {
      next() {
        if (current) {
          const value = current.value;
          current = current.next;
          return { value, done: false };
        }
        return { value: undefined, done: true };
      }
    };
  }
}

const list = new LinkedList();
list.push(1).push(2).push(3).push(4);

for (const val of list) {
  process.stdout.write(val + " ");   // 4 3 2 1
}

console.log([...list]);   // [4, 3, 2, 1]
const [first, second] = list;
console.log(first, second);  // 4 3
```

14.2 Generator Methods in Classes

```
class InfiniteCounter {
  constructor(start = 0, step = 1) {
    this.start = start;
    this.step  = step;
  }

  // Generator method — yields values one at a time
```

```
  * [Symbol.iterator]() {
    let current = this.start;
    while (true) {
      yield current;
      current += this.step;
    }
  }

  take(n) {
    const result = [];
    for (const val of this) {
      result.push(val);
      if (result.length === n) break;
    }
    return result;
  }
}

const evens = new InfiniteCounter(0, 2);
console.log(evens.take(5));   // [0, 2, 4, 6, 8]

const odds = new InfiniteCounter(1, 2);
console.log(odds.take(5));    // [1, 3, 5, 7, 9]
```

Chapter 15: Proxy & Reflect

A Proxy wraps an object and lets you intercept and customise fundamental operations — property access, assignment, deletion, function calls, and more. Reflect provides methods that mirror the same operations, making it easy to forward intercepted operations.

15.1 Validation with Proxy

```
function createValidatedObject(schema) {
  const data = {};

  return new Proxy(data, {
    set(target, key, value) {
      if (schema[key]) {
        const { type, min, max } = schema[key];
        if (typeof value !== type)
          throw new TypeError(`${key} must be a ${type}`);
        if (min !== undefined && value < min)
          throw new RangeError(`${key} must be >= ${min}`);
        if (max !== undefined && value > max)
          throw new RangeError(`${key} must be <= ${max}`);
      }
      return Reflect.set(target, key, value);  // perform the set
    },

    get(target, key) {
      if (key in target) return Reflect.get(target, key);
      throw new ReferenceError(`Property "${key}" does not exist.`);
    }
  });
}

const person = createValidatedObject({
  name: { type: "string" },
  age:  { type: "number", min: 0, max: 150 }
});

person.name = "Priya";    // OK
person.age  = 25;         // OK
console.log(person.name); // "Priya"

// person.age = "twenty";  // TypeError: age must be a number
// person.age = -5;        // RangeError: age must be >= 0
// person.email;           // ReferenceError: Property "email" does not exist
```

Chapter 16: Advanced "this" Binding

Understanding how "this" is determined is crucial in JavaScript OOP. The value of "this" depends on HOW a function is called, not where it is defined (except for arrow functions).

16.1 The Four Rules of "this"

1. Default Binding

In a regular function call (not a method), this is the global object (window in browsers, global in Node). In strict mode, it is undefined.

2. Implicit Binding

When a function is called as a method of an object (obj.method()), this is the object to the left of the dot.

3. Explicit Binding

Using call(), apply(), or bind() to explicitly set what this should be.

4. new Binding

When called with new, this is the newly created object.

```
class Timer {
  constructor(label) {
    this.label   = label;
    this.elapsed = 0;
  }

  // Arrow function captures "this" from constructor scope
  start() {
    const tick = () => {    // arrow — "this" is the Timer instance
      this.elapsed++;
      console.log(`${this.label}: ${this.elapsed}s`);
    };
    return setInterval(tick, 1000);
  }
}

// bind() explicitly fixes "this"
function logName() {
  return this.name;
}

const user1 = { name: "Alice" };
const user2 = { name: "Bob" };

const logAlice = logName.bind(user1);
const logBob   = logName.bind(user2);
```

```
console.log(logAlice());   // "Alice"
console.log(logBob());     // "Bob"

// call() and apply() — call immediately with explicit this
console.log(logName.call(user1));           // "Alice"
console.log(logName.apply(user2));          // "Bob"

// apply() takes arguments as an array
function greet(greeting, punctuation) {
  return `${greeting}, ${this.name}${punctuation}`;
}
console.log(greet.call(user1, "Hello", "!"));     // "Hello, Alice!"
console.log(greet.apply(user2, ["Hi", "?"]));     // "Hi, Bob?"
```

Chapter 17: Method Chaining & Fluent Interfaces

Method chaining allows you to call multiple methods in sequence on the same object. This creates a "fluent interface" — code that reads almost like natural language. The key is returning "this" from each method.

```
class Pizza {
  #size  = "medium";
  #crust = "thin";
  #toppings = [];
#extras = [];

  setSize(size)    { this.#size = size; return this; }
  setCrust(crust)  { this.#crust = crust; return this; }

  addTopping(topping) {
    this.#toppings.push(topping);
    return this;
  }

  addExtra(extra) {
    this.#extras.push(extra);
    return this;
  }

  build() {
    return {
      size:     this.#size,
      crust:    this.#crust,
      toppings: this.#toppings,
      extras:   this.#extras,
      summary: `${this.#size} pizza on ${this.#crust} crust`,
    };
  }
}

// Fluent interface — reads naturally
const myOrder = new Pizza()
  .setSize("large")
  .setCrust("thick")
  .addTopping("mozzarella")
  .addTopping("mushrooms")
  .addTopping("olives")
  .addExtra("extra cheese")
  .build();

console.log(myOrder.summary);
// "large pizza on thick crust"
console.log(myOrder.toppings);
// ["mozzarella", "mushrooms", "olives"]
```

Chapter 18: Best Practices & Quick Reference

18.1 Best Practices

- **Use classes over constructor functions:** Classes are cleaner, more readable, and the modern standard.
- **Keep classes small and focused:** A class should have one responsibility (Single Responsibility Principle).
- **Use private fields (#) for encapsulation:** Truly hide internal state rather than relying on naming conventions.
- **Prefer composition over deep inheritance:** Inheritance hierarchies deeper than 2-3 levels become hard to maintain.
- **Return "this" for method chaining:** When it makes sense, enables clean fluent interfaces.
- **Avoid using arrow functions as methods:** Arrow functions do not bind their own "this" and cannot be used as constructors.
- **Validate inputs in constructors:** Throw errors early with descriptive messages rather than letting objects enter invalid states.

18.2 OOP Concept Quick Reference

Concept	Keyword / Syntax	Purpose
Class	`class MyClass { }`	Define a blueprint for objects
Constructor	`constructor(...args)`	Initialize object when new is called
Instance Method	`myMethod() { }`	Method shared via prototype
Static Method	`static myMethod()`	Method on the class, not instances
Getter / Setter	`get x() / set x(v)`	Computed properties with logic
Private Field	`#fieldName`	Truly private class state
Private Method	`#methodName()`	Truly private class behaviour
Inheritance	`class Child extends Parent`	Inherit and extend a parent class
Super Constructor	`super(...args)`	Call parent constructor
Super Method	`super.method()`	Call parent method from child
instanceof	`obj instanceof Class`	Check object type in hierarchy
Object.create()	`Object.create(proto)`	Create object with specific prototype
Object.freeze()	`Object.freeze(obj)`	Make object fully immutable
Symbol.iterator	`[Symbol.iterator]()`	Make class iterable (for...of)

Concept	Keyword / Syntax	Purpose
Proxy	`new Proxy(target, handler)`	Intercept object operations

End of Document